

The Local Harness Guide

How to run a free local model as a real coding agent, and how to use Claude or Codex to train it into one.

COMPANION GUIDE | LOCAL AI CARE PACKAGE

Mark Kashef

<https://www.skool.com/earlyaidopters/about>

HOW TO USE THIS GUIDE

Four sections, in order. Section 1 and 2 are setup and take an afternoon. Section 3 is the actual work and never really finishes. Section 4 is the part most people skip and then get angry about.

The claim behind this guide

A free local model, in a good harness, produced four complete websites with zero tool errors across four headless runs. The same model in a stock harness could not finish one. Nothing about the model changed between those two outcomes.

What a harness actually is

Everything sitting between the model and your machine. The instructions it always reads, the tools it can reach, the rules that stop it, and the checks that decide whether it succeeded.

Most of what people call "the model is not good enough" is a harness that never told the model what done means.

The four sections

Section 1	Get the models. LM Studio, sizing for your hardware, and the one setting that breaks everything.
Section 2	Get the harness. Installing pi, pointing it at LM Studio, skills versus extensions, headless mode.
Section 3	Train it with Claude or Codex. The babysitting pattern and the five fixes it produced.
Section 4	The honest limits. What stays cloud, render economics, and when to just pay.

If you read one section, read 3. If you apply one thing from it, apply the goal gate.

1.1 Install LM Studio

LM Studio is the easiest way to run a local model behind an OpenAI-compatible API, which is what every agent harness expects to talk to. Download it, open it, and go to the Discover tab.

1.2 Pick a model your hardware can hold

```
Maxed-out machine (96GB+ unified memory)
A large mixture-of-experts model, e.g. Qwen3.5 122B A10B (4-bit MLX)
This is the tier that completes full builds.
```

```
Mid hardware (32 to 64GB)
A 24B to 27B class model, e.g. Qwen3.6 27B or Mistral Small 24B
Good for single edits and small features. Not full one-shot builds.
```

```
Lighter hardware (16GB)
7B to 14B class. Narrow, well-specified tasks only.
```

Two properties matter more than raw size.

Vision capability. The harness is going to screenshot its own work and look at it. A model that cannot take an image loses the single most valuable check you can give it. Check the model card for image input before downloading.

A real context window. Long builds die on context, not intelligence. Prefer models offering 128k or more.

On Apple Silicon, prefer MLX builds. They are meaningfully faster than the GGUF equivalent on the same machine.

1.3 Raise the context window

LM Studio defaults the context window to 8192 tokens regardless of what the model supports.

This is the most common reason a local coding agent behaves like it has brain damage. Eight thousand tokens is a few files. The agent reads a file, reads another, and the first has already fallen out of its memory. The symptoms look like stupidity but are amnesia.

You will also see this error, which confuses everyone the first time:

```
tokens to keep > context length
```

That means the harness asked to retain more context than the server was configured to hold. It is a settings problem, not a model problem.

Fix it in the model's load settings. Raise the context length to what the model actually supports, 131072 or 262144 for the models above.

Then verify it stuck

Quit LM Studio completely, reopen it, load the model, and check the setting again. It does not always persist, and a silently reverted context window sends you straight back to debugging phantom stupidity.

1.4 Start the server

In the Developer tab, start the local server. It listens on port 1234 by default and speaks the OpenAI completions API. Confirm it is up:

```
curl -s http://localhost:1234/v1/models
```

You should get JSON listing the loaded model. That string is the model id you need next, and it must match exactly.

2.1 Install pi

pi is an open coding agent from pi.dev. Install it and run **pi** in a project folder. Any agent supporting custom instructions, tools and skills works for this guide, pi is just the one this was built on.

2.2 Point it at LM Studio

Configuration lives in **models.json** in your pi agent directory. A working example, genericized:

```
{
  "providers": {
    "lmstudio": {
      "baseUrl": "http://localhost:1234/v1",
      "api": "openai-completions",
      "apiKey": "lmstudio",
      "compat": {
        "supportsDeveloperRole": false,
        "supportsReasoningEffort": false,
        "maxTokensField": "max_tokens"
      },
      "models": [
        {
          "id": "your-model-id-from-lm-studio",
          "name": "Your Model (LM Studio, MLX)",
          "reasoning": true,
          "input": ["text", "image"],
          "contextWindow": 262144,
          "maxTokens": 32768
        }
      ]
    }
  }
}
```

Four fields people get wrong

- **apiKey** must be present but the value is ignored. LM Studio does not check it.
- **id** must match the server's model id exactly, not the display name.
- **input** must include image or the harness will not send screenshots, silently.
- **contextWindow** must match LM Studio. If they disagree the smaller wins, unannounced.

2.3 What ships in the box

A stock harness gives you roughly six things. Knowing what you already have stops you rebuilding it.

Part	What it does
background	Runs long processes without blocking the session
compactor	Summarizes old context when the window fills
guard	Confirms before destructive actions
subagent	Delegates isolated subtasks so their context does not pollute yours
todo	The task list the agent plans against
watchdog	Notifies when a run has gone off the rails

That is a competent starting point and nowhere near enough for a free local model. Everything in Section 3 is what gets added on top.

2.4 Skills versus extensions, in plain speak

A skill is knowledge. A markdown file teaching the agent how to do a kind of work. When to use it, the steps, the references, the gotchas. Skills change what the agent knows.

An extension is machinery. Code that adds a tool or enforces a rule, running whether the model cooperates or not. Extensions change what the agent can and cannot do.

This distinction decides where every fix goes, and getting it wrong is the most common mistake. If the agent needs to know something, that is a skill. If the agent needs to be stopped from something, that must be an extension, because a model capable of ignoring an instruction eventually will.

2.5 Reload after changes

pi loads skills and extensions at startup. After adding or editing either, run **/reload** or restart. Confirm with **/skills**. Editing a skill and wondering why nothing changed is a rite of passage worth skipping.

2.6 Running a whole job unattended

```
pi -p "$(cat brief.md)"
```

This is how you run a full build unattended, and how you run the exam in Section 3.7.

The gotcha

Slash commands do not exist in headless mode. If your interactive workflow sets gates with `/gate npm run build`, that command is not available to a headless run. It fails silently, meaning your unattended build has no completion gate at all, which is exactly when you need one most.

The fix is to carry the gates in the brief itself, as instructions:

```
Before declaring this task complete you MUST verify all of the following.  
A failing check means the job is NOT done, no matter how good the output looks.
```

1. `npm run build` exits successfully
2. `grep -r '{{' index.html` returns nothing
3. `DESIGN.md` exists and records the variation picks

```
Run all three yourself and report the result of each.
```

Every skill written for headless use should carry its gates this way, in its own body.

SECTION 3 · TRAIN IT WITH CLAUDE OR CODEX

The reason a free local model can build a real website is not prompting. It is that a frontier model watched it fail, repeatedly, and every failure became a permanent part of the harness.

3.1 The babysitting pattern

- Give your local model a real task. Not a toy. A real build with a real deliverable.
- Have Claude Code or Codex tail the session log while it runs. Watching, not doing the work.
- When it catches a failure, do not fix it in the chat. Convert it into a permanent harness part.
- Repeat until the failures stop being about the harness and start being about the task.

The setup is simply pointing the frontier model at the log:

```
tail -f ~/.pi/agent/sessions/<session-id>.log
```

Then the brief for the watcher:

```
You are watching a local 4-bit model run a real build through this log.  
Do NOT do the work and do NOT fix anything yourself.
```

```
Watch for: repeated commands, silent retries, edits based on stale reads,  
declarations of success without verification, and drift from the brief.
```

```
When you see one, tell me three things.
```

1. What went wrong, in one sentence.
2. Whether the fix belongs in an extension (a hard block), a skill (knowledge), or the always-read instructions file (a standing rule).
3. The smallest change that prevents it recurring for every future run.

That third question is the whole technique. A frontier model is very good at spotting the difference between "this run went wrong" and "this class of run will always go wrong."

Why the frontier model and not you. It reads a thousand lines of log in seconds and knows what good looks like. You will scroll past the exact moment the agent started looping.

The rule that makes it compound. Every catch becomes a permanent harness part, never a correction typed into the chat. A correction helps for one session. A harness addition helps forever. Seven of the extensions in this setup came from exactly this loop, and by the end the local model was completing full builds with zero tool errors.

3.2 · THE 35-MINUTE HANG, BECOMING A NETWORK GUARD

What happened

The agent tried to install a package with no connectivity. It hung, timed out, tried a different package manager, hung again. Thirty-five minutes of a session spent achieving nothing.

Why an instruction was not enough

"Check connectivity first" is exactly the kind of rule a small model drops under load.

The permanent fix

An extension that checks connectivity before any install or fetch and fails immediately:

```
OFFLINE: no connectivity. This command was blocked rather than hung.  
Do not retry it and do not try another package manager.  
Work with what is already on the machine.
```

The message matters as much as the block. It tells the agent what to do instead, otherwise a blocked command just becomes a different loop.

The standing rule that goes in the instructions file alongside it:

```
This is a laptop. Connectivity is NOT guaranteed. If a command is blocked as  
OFFLINE, do not retry it and do not try another package manager. Before  
installing anything, check it isn't already present.
```

3.3 · THE EDIT-RETRY LOOP, BECOMING A FRESHNESS BLOCK

What happened

The agent read a file, did several other things, then edited it from memory. The file had changed. The edit landed on stale content and quietly reverted earlier work. Then it edited again to fix damage it could not see.

The permanent fix

A hard block. Any file whose content changed since the agent last read it cannot be edited until it is re-read. Not a warning, a refusal, because this failure is invisible in the transcript and the agent has no way to notice it.

The companion rule for context generally:

```
Important facts from tool output (versions, paths, error messages) should be  
restated in your own text. Old tool results may be dropped from context later,  
and you will confidently misremember them.
```

3.4 · INVISIBLE LOOPING, BECOMING THE GOAL GATE

What happened

The agent ran the same failing command a dozen times with small variations, then declared the task complete. The build was broken. It said it was done.

This is the highest-leverage fix in the entire guide.

The permanent fix

Completion conditions defined before the work starts, as commands that pass or fail:

```
/gate npm run build
/gate ! grep -rq '{{' index.html
/gate test -f DESIGN.md
```

The harness runs these on any attempt to declare completion and refuses the declaration until all pass.

Note what they share. They are mechanical. "The site looks good" is not a gate. "npm run build exits zero" is.

Loop detection belongs with it. If the same command fails twice with the same error, stop and rethink rather than trying a third time.

If you take one thing from this guide, take the goal gate. It converts an agent that claims success into one that earns it.

3.5 · STYLED BUT NEVER ADDED, THE MARKUP CONTRACT CHECK

What happened, and this one fooled everything

Building a cabin-rental site, the model wrote a beautiful banner section in CSS, class names and all. It never added the matching divs to the HTML. The build passed. No console errors. The section rendered as cream text on a cream background, completely invisible. The model reported success, and it was not lying. Every check it had said yes.

The permanent fix

```
for cls in $(grep -o '^\.[a-z][a-z0-9_-]*' src/style.css | sort -u | tr -d '.'); do
  grep -q "class=\"[^\"]*$cls" index.html || echo "MISSING IN HTML: .$cls"
done
```

Every miss gets investigated. The only legitimate ones are classes created at runtime by JavaScript.

The general principle is bigger than the script. Any time an agent writes across two files that must agree, there is a mechanical check proving they do, and it belongs in the harness rather than in your review. Ask what your builds silently allow to disagree.

3.6 · THE BROWSER SELF-CHECK WITH A FORCED VERDICT

If your agent can screenshot, it is a vision model, and it can grade its own output. Text-only reasoning about a page it has never seen is worthless, and it will still sound confident.

The pattern that works, called with an explicit expectation and returning a forced verdict:

```
browser_check http://localhost:5173
  expect: navigation bar, serif hero headline, hero media frame visible
  scrollTo: 0
  mobile: false

-> VERDICT: PASS or FAIL
```

Run it at several scroll positions (0, 0.3, 0.6, 1.0) and once at mobile width. Loop until every verdict passes.

The forced verdict is the trick. Without it the model writes a paragraph describing the screenshot and moves on. PASS or FAIL with nothing in between forces a decision it then has to act on.

Two rules that keep it honest

Never dismiss a console error, 404, or failed request as "unrelated" or "pre-existing". Every asset the page references is yours to prove working (curl it, expect 200) or to fix, before calling the work done.

Judge the design honestly. Generic hero-and-cards output is a FAIL even if it renders correctly.

And the rule for when you say it looks the same

If you tell the agent nothing changed and it immediately makes another edit, you now have two changes and no idea which did what.

On "looks the same", do NOT stack another edit. First prove the previous change reached the browser: right file loaded, build rebuilt, cache busted, styles not overridden. Then verify visually that pixels actually changed.

3.7 · THE EXAM

Baking in fixes feels productive. It is not evidence. The test is whether they hold without you.

The exam pattern

After a round of harness fixes, run a completely fresh one-shot on a new brief. New session, no prior context, headless, and no touch-ups. Then grade it:

- Did it finish without intervention?
- Did the specific failures you fixed recur?
- Did the gates catch anything, or did it pass them honestly?
- Tool errors, count them.

The bar that matters is zero touch-ups. A build you rescued twice tells you nothing about the harness.

Run the exam on a genuinely new subject, not a rerun of the brief you have been debugging with. An agent that succeeds on the fifth attempt at the same site has memorized nothing, but you have tuned the harness to one input.

3.8 · THE DESIGN-HISTORY REGISTRY

A subtle failure you only see across runs. Ask the same agent for three sites and it gives you the same site three times, because nothing tells it what it already built.

Have each build append one line to a registry, then have the skill read that registry before choosing:

```
2026-08-09 | COVE | site-6 | Fraunces+Inter, taupe, magnetic buttons, char cascade  
2026-08-09 | HEARTH | site-7 | Cormorant+Sora, forest green, film grain, tilt cards
```

The rule in the skill is that shipping the same combination twice is a failure. Writing a design document inside the project is not enough, that file is invisible to the next run. The registry is the memory.

3.9 · THE TOKEN DIET

Every skill, tool and instruction you load costs context the model needs for the work. On a local model this bites much sooner than expected, and the symptom is confusing: quality degrades midway through long jobs for no visible reason.

- Move rarely-used skills behind a small router skill. Only the summary loads, the body is pulled on demand.
- Keep the always-read instructions file short and rule-shaped, not prose.
- Push deep reference material into files opened when needed, not the system prompt.
- Cap the agent's own narration. Verbose commentary between tool calls is context spent on nothing.

Routing deserves its own note

A large model reads twelve skill descriptions and picks correctly. A small model picks whichever it saw last, or invents a hybrid. Route by explicit priority rather than the model's judgment.

When a request for a clean product page kept being answered with WebGL and a scroll-scrubbed video, the fix was not a better description. It was ranking the editorial skill above the immersive one.

Have each skill open by naming the cases belonging to a different skill and sending the agent there. Redundant routing is cheap, a wrong archetype costs an entire build.

Everything above is real and reproducible. Here is what it does not do.

4.1 What stays in the cloud

Large video models do not run on your laptop. The cinematic model used for the hero footage in the video is cloud only, and no amount of local setup changes that. Anyone claiming a fully local cinematic video pipeline at that quality is skipping the part where they paid.

The honest local path for video is 720p-class open models. They work, they are free, they are visibly a tier below.

4.2 The render economics

```
Local 720p open video model 30 minutes to 3 hours per clip
Overnight run, realistically one hero clip, maybe two
Cloud cinematic render minutes, and a few dollars
```

A text model on a good laptop is genuinely competitive with paid tools. A video model is not close.

Your laptop is a great brain and a poor render farm.

Budget by hours, not dollars. The question is not whether you can afford it, it is whether you can wait.

4.3 Closing the gap with free upscalers

The move that makes the local path viable. Generate at the resolution your machine can actually reach, then upscale with a free tool. Real-ESRGAN at 4x takes 768p footage to something that holds up full-screen, and frame interpolation smooths the result.

You upgrade the pixels, not the machine.

Four gotchas that cost real time

- Point it at the models directory explicitly with -m, it will not find them otherwise.
- Feed it JPG, not PNG. PNG input fails in ways that do not explain themselves.
- Run it in the foreground for GPU acceleration. Backgrounded, it silently falls back to CPU.
- Upscale frame by frame, then reassemble. Whole-video upscaling is a memory trap.

SECTION 4 · WHEN TO JUST PAY

Be honest with yourself about which of these you are.

Local is right when

- You build often enough that subscriptions add up.
- You want the work to survive someone else's pricing change.
- You can run things overnight.
- Your machine is already strong.

Paying is right when

- You need one site this afternoon.
- You need cinematic video regularly.
- Your hardware is mid.
- Your time is worth more than the setup.

The setup in this guide took weeks of failures to arrive at. It is now reusable forever and costs nothing per run. That trade is good if you are going to keep using it, and bad if this is a one-off.

Where to start

If you do one thing, add the goal gate. Define done as commands that pass or fail and refuse the declaration until they do. An afternoon of work, and it converts an agent that claims success into one that earns it.

Then add the browser check, so the agent sees its own output.

Then start the babysitting loop and let it tell you which of the rest you need. Your failures will not be identical to mine, and a harness built from someone else's failure list is just more context you are paying for.

GO DEEPER

Want to build the whole local stack, not just the websites?

Inside the Early AI Dopters community, you get the full **LIVING** local AI build-out, taught from zero:

- Running open source models on your own hardware, from picking the model to wiring the harness
- The exact chain behind this care package, video model to agent to skill, walked through step by step in a dedicated module
- Direct access to me and my team of coaches when your build stalls
- A working group of builders shipping their own sites and their own harnesses

JOIN HERE

<https://www.skool.com/earlyaidopters/about>

The Local Harness Guide · companion to the Local AI Care Package